

SparseTap — Report

KRAFTON AI R&D Hackathon · Round 1 · Day 2 · Team #179

1. Problem Analysis

SparseTap asks us to recover S hidden offsets $d_1 < d_2 < \dots < d_s$ ($S \leq 16$, $d_s \leq 64$) from 2,000 noisy binary sequences of length 256, where each bit after position $W = d_s$ satisfies:

$$\text{seq}[n] = \text{seq}[n-d_1] \oplus \text{seq}[n-d_2] \oplus \dots \oplus \text{seq}[n-d_s] \oplus e[n], \quad e[n] \sim \text{Bernoulli}(0.2)$$

This is the Sparse Learning Parity with Noise (LPN) problem — equivalent to a Fast Correlation Attack on a stream cipher's LFSR (Meier & Staffelbach, 1989).

Information-theoretic feasibility. Each equation carries $1 - H(0.2) \approx 0.278$ bits of information about the secret. With $2,000 \times 192 = 384,000$ equations and 64 unknowns, we have $\sim 106,700$ bits of usable information — vastly more than the 64 bits needed.

The core difficulty: Piling-up Lemma. In $\text{GF}(2)$, correlation of XOR of k independent ϵ -biased bits decays as $(1-2\epsilon)^k$. For a single offset d , the observed signal is $\text{seq}[n] \oplus \text{seq}[n-d] = (\text{XOR of } S-1 \text{ other offsets}) \oplus \text{noise}$. When $S = 8$, the per-offset bias is $0.6^8 \approx 0.017$ — indistinguishable from random with our sample size. This is why greedy, LASSO, and single-offset correlation methods structurally fail: **there is no partial credit in $\text{GF}(2)$.**

2. Approach 1: BKW + Hierarchical FWHT (Main — Success)

We used the Blum–Kalai–Wasserman (BKW) algorithm, the standard attack on LPN.

Step 1: Build equation system. From 2,000 sequences $\times$ 192 positions = 384,000 equations of the form $\mathbf{A} \cdot \mathbf{s} = \mathbf{b} \pmod{2}$ with 20% noise, where $\mathbf{s} \in \{0,1\}^{64}$ is the secret offset indicator vector.

Step 2: BKW dimension reduction. We partition the 64 columns into blocks of 16. For each block, we group equations by their values in that block and XOR pairs within each group, zeroing out those 16 columns. After each round, noise roughly doubles: round 1 noise ≈ 0.32 , round 2 noise ≈ 0.435 (via $P_{\text{new}} = 2p(1-p)$). This trades equations for reduced dimensionality.

Step 3: FWHT subspace search. After 2 BKW rounds, only 32 columns remain. We perform the Fast Walsh–Hadamard Transform on the reduced system to find the 32-bit partial secret maximizing the bias. FWHT searches all 2^{16} candidates per 16-bit sub-block in $O(16 \cdot 2^{16})$ time.

Step 4: Hierarchical back-substitution. We substitute the recovered bits back into the round-1 equations and run FWHT on the next 16-bit block, then repeat for the first block. This avoids ever searching 2^{32} or 2^{64} spaces.

Result: Offsets **[5, 14, 21, 29, 36, 42, 50, 57]** ($S = 8$, $W = 57$) recovered in **0.78 seconds** on CPU. Training accuracy = **79.97%** (expected 80.0%). Bootstrap 95% CI: [79.8%, 80.2%]. Leave-one-out: removing any single offset drops accuracy to $\sim 50\%$. Adding any extra offset also drops to $\sim 50\%$.

3. Approach 2: Meet-in-the-Middle (Independent Success)

Brute-force over $C(64,8) \approx 4.4$ billion offset combinations is infeasible. Meet-in-the-Middle (MitM) splits the search space to reduce complexity dramatically.

Step 1: Split. Partition the 64 candidate positions into LEFT (1–32) and RIGHT (33–64). Assuming $S = 8$ with a (4,4) split, enumerate $C(32,4) = 35,960$ masks per side — only 71,920 total.

Step 2: Precompute. For each left mask, compute $\text{adjusted_output} = \text{output} \oplus \text{parity_L}$ for all 384,000 equations. Store as a lookup table keyed by the left-side parity pattern.

Step 3: Match. For each right mask, compute parity_R and find the left mask whose adjusted output maximally correlates with parity_R . The (left, right) pair with the highest combined correlation reveals the true offsets.

Result: Offsets **[5, 14, 21, 29, 36, 42, 50, 57]** independently recovered in **3.19 seconds**. Training accuracy = **79.99%**. This provides independent confirmation of the BKW result. Complexity reduction: $C(64,8) \approx 4.4\text{B} \rightarrow 2 \times C(32,4) \approx 72\text{K}$ — a **60,000 $\times$ speedup**. In general, one should also try (3,5) and (2,6) splits for robustness when the true split is unknown.

4. Approach 3: Gradient Descent with Continuous Relaxation (Numerical Method)

Formulation. Replace $\mathbf{s} \in \{0,1\}^{64}$ with continuous weights via sigmoid: $w_d = \sigma(\alpha_d)$. Map $\{0,1\} \rightarrow \{\pm 1\}$ so XOR becomes multiplication, giving a differentiable loss. Optimize with Adam (lr=0.01, 5000 steps) with temperature annealing.

Result: This approach failed. All 64 weights converged to ~0.45, producing no differentiation. The Piling-up Lemma causes **gradient starvation**: $\partial L / \partial \alpha_d$ involves a product of $(S-1)$ near-zero terms, making gradients exponentially small in S . The loss landscape has 2^{64} local minima of nearly equal depth — a structural limitation of gradient methods on XOR parity problems, not a hyperparameter issue. An MLP with a 32-unit hidden layer also failed for the same reason.

5. Failed Attempts

Greedy single-offset correlation. All 64 offsets showed accuracy $\approx 50\%$ (max $|z| < 2$). Per the Piling-up Lemma, single-offset bias is $0.6^8 \approx 0.017$ — undetectable with 2,000 sequences.

Exhaustive search ($S = 1..6$). All $C(64,S)$ combinations for $S \leq 6$ gave ~50% accuracy, confirming $S \geq 7$.

RANSAC (162K trials). $P(\text{all 64 equations noise-free}) = 0.8^{64} \approx 6 \times 10^{-7}$. No solution found in 55 seconds.

Bootstrap GF(2) majority vote (1000 trials). A single flipped bit in GF(2) Gaussian elimination propagates to corrupt the entire solution. Majority voting across corrupted solutions produced only noise.

6. Results

Item	Value
Offsets	[5, 14, 21, 29, 36, 42, 50, 57]
S / W	8 / 57
Training accuracy	79.97% (expected 80%)
Bootstrap 95% CI	[79.8%, 80.2%]
Successful methods	BKW + FWHT (0.78s), Meet-in-the-Middle (3.19s)
Independent verification	Two methods converge on identical offsets

Answer (192 bits, positions 64–255):

```
111101101100111001011111000011010101101100100011000100000001000011100000
110101010000010101011111000111001010001011100010110100010111111101100111
001101110110010001110110110010011001001011111001
```